

Q1. Caesar Cipher

```
[4] 9s
def caesar_encrypt(text, key):
    result = ""
    for ch in text:
        if ch.isalpha():
            shift = key % 26
            if ch.isupper():
                result += chr((ord(ch) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result

def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)

text = input("Enter Plain Text: ")
key = int(input("Enter Key (1-25): "))

cipher = caesar_encrypt(text, key)
print("Encrypted Text:", cipher)
print("Decrypted Text:", caesar_decrypt(cipher, key))

... Enter Plain Text: HELLO
Enter Key (1-25): 3
Encrypted Text: KHOOR
Decrypted Text: HELLO
```

Q2. Monoalphabetic Substitution Cipher

```
[5] 18s
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
key = "QWERTYUIOPASDFGHJKLZXCVBNM"

def encrypt(text):
    result = ""
    for ch in text.upper():
        if ch in alphabet:
            index = alphabet.index(ch)
            result += key[index]
        else:
            result += ch
    return result

def decrypt(text):
    result = ""
    for ch in text.upper():
        if ch in key:
            index = key.index(ch)
            result += alphabet[index]
        else:
            result += ch
    return result

plain = input("Enter Plain Text: ")

cipher = encrypt(plain)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher))

... Enter Plain Text: HELLO
Encrypted Text: ITSSG
Decrypted Text: HELLO
```

Q3. Playfair Cipher

```
[7] 17s
def encrypt(text, matrix):
    text = prepare_text(text)
    cipher = ""

    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]
        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)

        if r1 == r2:
            cipher += matrix[r1][(c1+1)%5]
            cipher += matrix[r2][(c2+1)%5]
        elif c1 == c2:
            cipher += matrix[(r1+1)%5][c1]
            cipher += matrix[(r2+1)%5][c2]
        else:
            cipher += matrix[r1][c2]
            cipher += matrix[r2][c1]

    return cipher

key = input("Enter Keyword: ")
text = input("Enter Plain Text: ")

matrix = generate_matrix(key)

print("Playfair Matrix:")
for row in matrix:
    print(row)

print("Encrypted Text:", encrypt(text, matrix))

... Enter Keyword: MONARCHY
Enter Plain Text: WORLD
Playfair Matrix:
['M', 'O', 'N', 'A', 'R']
['C', 'H', 'V', 'B', 'D']
['E', 'F', 'G', 'I', 'K']
['L', 'P', 'Q', 'S', 'T']
['U', 'V', 'W', 'X', 'Z']
Encrypted Text: VNMTBZ
```

Q4. Polyalphabetic (Vigenère) Cipher

```
[8] 33s
def encrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text.upper():
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
            j += 1
        else:
            result += ch
    return result

def decrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text.upper():
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65
            result += chr((ord(ch) - 65 - shift) % 26 + 65)
            j += 1
        else:
            result += ch
    return result

plain = input("Enter Plain Text: ")
key = input("Enter Key: ")

cipher = encrypt(plain, key)

print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, key))

... Enter Plain Text: NETWORK
Enter Key: DATA
Encrypted Text: QEMMRD
Decrypted Text: NETWORK
```

Q5. Affine Caesar Cipher

```
a = int(input("Enter value of a: "))
b = int(input("Enter value of b: "))

if gcd(a, 26) != 1:
    print("Invalid value of a. It must be coprime with 26.")
else:
    cipher = encrypt(plain, a, b)
    print("Encrypted Text:", cipher)
    print("Decrypted Text:", decrypt(cipher, a, b))
```

```
... Enter Plain Text: CYBER
Enter value of a: 4
Enter value of b: 7
Invalid value of a. It must be coprime with 26.
```